DSA in Go

NeetCode 150 · Arrays & Strings

Complete Interview Preparation Guide

28 Problems

4 Categories

Both BF + Optimal

Go Language

Category	Problems	Key Technique	Complexity Gain
Arrays & Hashing	9	Hash Maps	$O(n^2) \rightarrow O(n)$
Two Pointers	5	Convergent Pointers	$O(n^2) \rightarrow O(n)$
Sliding Window	6	Window Reuse	$O(n^2) \rightarrow O(n)$
Math & Geometry	8	Mathematical Tricks	O(1) space

How to Use This Guide

- **Study Order:** Arrays & Hashing → Two Pointers → Sliding Window → Math & Geometry
- **Per Problem:** Read the analogy first → Understand brute force → Study the optimization insight → Memorize the formula
- **Interview Prep:** For each problem, practice explaining the 'Why' before the 'How'
- **Complexity:** Always state both Time AND Space complexity. Mention trade-offs.
- **Edge Cases:** Empty input, single element, all same elements, negatives, overflow

■ Arrays & Hashing

1 Contains Duplicate

Arrays & Hashing

Easy

Problem: Given an integer array nums, return true if any value appears at least twice.

■ Constraints

1 <= nums.length <= 10^5 | -10^9 <= nums[i] <= 10^9

■ Real-World Use Cases

Banking fraud detection (duplicate transactions), user deduplication in databases, cache invalidation checks.

■ What If This Pattern Didn't Exist?

Without hash sets we'd sort O(n log n) or do O(n^2) nested comparisons — impractical for millions of records.

■ Memory Analogy

Think of a bouncer with a guest list clipboard. As each person arrives, he checks the clipboard. If the name is already there — duplicate! If not, he

■ Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-1:
   FOR j = i+1 to n-1:
      IF nums[i] == nums[j] RETURN true
RETURN false
```

Go — Brute Force

```go
func containsDuplicate(nums []int) bool {
	for i := 0; i < len(nums); i++ {
		for j := i + 1; j < len(nums); j++ {
			if nums[i] == nums[j] {
				return true
			}
		}
	}
	return false
}
```

■ Optimized Approach

Pseudo Code

```
CREATE empty hash set
FOR each num in nums:
   IF num in set: RETURN true
   ADD num to set
RETURN false
```

Go — Optimized

```go
func containsDuplicate(nums []int) bool {
	seen := make(map[int]bool)
	for _, num := range nums {
		if seen[num] {
			return true
		}
		seen[num] = true
	}
	return false
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(n)

Formula	Memory Tip
Hash lookup = O(1) avg. Trade SPACE for TIME.	"Seen before? Return true. New? Add to set." — One pass, one set.

Interview Tips & Edge Cases

Ask: 'Can I use extra space?' Mention sorting as an O(n log n) alternative if space is constrained. Edge case: empty array → false.

2

Valid Anagram

Arrays & Hashing

Easy

Problem: Given two strings s and t, return true if t is an anagram of s (same chars, same frequencies).

Constraints

1 <= s.length, t.length <= 5*10^4 | lowercase English letters only

Real-World Use Cases

Plagiarism detection (rearranged words), search engine synonym grouping, DNA sequence comparison.

What If This Pattern Didn't Exist?

You'd compare every permutation — O(n! * n) — completely infeasible for long strings.

Memory Analogy

You have two bags of Scrabble tiles. Pour each bag into separate piles sorted alphabetically. If both piles look identical, they're anagrams. The fre

Brute Force Approach

Pseudo Code

```
SORT characters of s -> s_sorted
SORT characters of t -> t_sorted
RETURN s_sorted == t_sorted
```

Go — Brute Force

```go
import "sort"

func isAnagram(s string, t string) bool {
	if len(s) != len(t) { return false }
	rs, rt := []rune(s), []rune(t)
	sort.Slice(rs, func(i,j int) bool { return rs[i]<rs[j] })
	sort.Slice(rt, func(i,j int) bool { return rt[i]<rt[j] })
	return string(rs) == string(rt)
}
```

Optimized Approach

Pseudo Code

```
IF len(s) != len(t): RETURN false
CREATE freq map [26]int
FOR each char in s: freq[char-'a']++
FOR each char in t: freq[char-'a']--
FOR each count in freq:
   IF count != 0: RETURN false
RETURN true
```

Go — Optimized

```go
func isAnagram(s string, t string) bool {
	if len(s) != len(t) { return false }
	var freq [26]int
	for i := 0; i < len(s); i++ {
		freq[s[i]-'a']++
		freq[t[i]-'a']--
	}
	for _, c := range freq {
		if c != 0 { return false }
	}
	return true
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n log n)

Space: O(n)

OPTIMIZED

Time: O(n)

Space: O(1) — fixed 26-char array

Formula	Memory Tip

Anagram: freq(s) == freq(t). Count up in s, count down in t, all zeros = anagram.	'+1 for s, -1 for t'. All zeros at end = anagram. Like a balance scale.

■ Interview Tips & Edge Cases

Unicode follow-up: use map[rune]int instead of [26]int. Always check length first as early exit.

---

3 Two Sum

Arrays & Hashing

Easy

Problem: Given array nums and target, return indices of two numbers that add up to target.

■ Constraints

2 <= nums.length <= 10^4 | Exactly one solution exists | Can't use same index twice

■ Real-World Use Cases

Financial: find two transactions summing to a refund amount. Pair matching in recommendation systems. Database join optimization.

■ What If This Pattern Didn't Exist?

Brute force checks every pair O(n^2). For 10k elements that's 100M comparisons. Hash map reduces to 10k.

■ Memory Analogy

You're at a party looking for your dance partner. Instead of asking everyone 'Are you my complement?', you write your own number on a sticky not

■ Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-1:
   FOR j = i+1 to n-1:
      IF nums[i] + nums[j] == target:
         RETURN [i, j]
```

Go — Brute Force

```go
func twoSum(nums []int, target int) []int {
	for i := 0; i < len(nums); i++ {
		for j := i + 1; j < len(nums); j++ {
			if nums[i]+nums[j] == target {
				return []int{i, j}
			}
		}
	}
	return nil
}
```

■ Optimized Approach

Pseudo Code

```
CREATE map: value -> index
FOR i, num in nums:
   complement = target - num
   IF complement in map:
      RETURN [map[complement], i]
   map[num] = i
```

Go — Optimized

```go
func twoSum(nums []int, target int) []int {
      seen := make(map[int]int) // value -> index
      for i, num := range nums {
            complement := target - num
            if j, ok := seen[complement]; ok {
                  return []int{j, i}
            }
            seen[num] = i
      }
      return nil
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(n)

Formula	Memory Tip
complement = target - current. Look up complement in map BEFORE inserting current.	"What do I NEED? = target - me. Check the wall first, then post my note."

Interview Tips & Edge Cases

Sorted array variant → Two Pointers (Two Sum II). Multiple solutions → return all pairs. No extra space → sort + two pointers O(n log n).

4

Group Anagrams

Arrays & Hashing

Medium

Problem: Given array of strings, group all anagrams together and return the groups.

Constraints

1 <= strs.length <= 10^4 | 0 <= strs[i].length <= 100 | lowercase letters

Real-World Use Cases

Search engines grouping synonyms, autocomplete suggestions, dictionary word clustering, spell-check grouping.

■ What If This Pattern Didn't Exist?

Comparing every pair of strings for anagram property: O(n^2 * k) where k = string length.

■ Memory Analogy

Think of sorting your wardrobe. Each shirt has a 'tag' made by sorting its letters alphabetically. All shirts with the same tag go in the same drawer.

■ Brute Force Approach

Pseudo Code

```
FOR each pair (i,j) of strings:
   IF isAnagram(s[i], s[j]):
      group them together
// O(n^2 * k log k)
```

Go — Brute Force

```go
// O(n^2 * k) approach — check all pairs
func groupAnagrams(strs []string) [][]string {
	visited := make([]bool, len(strs))
	result := [][]string{}
	for i, s := range strs {
		if visited[i] { continue }
		group := []string{s}
		visited[i] = true
		for j := i+1; j < len(strs); j++ {
			if !visited[j] && isAnagramStr(s, strs[j]) {
				group = append(group, strs[j])
				visited[j] = true
			}
		}
		result = append(result, group)
	}
	return result
}
```

■ Optimized Approach

Pseudo Code

```
CREATE map: sorted_string -> []strings
FOR each str in strs:
   key = sort(str)  // canonical form
   map[key] = append(map[key], str)
RETURN values of map
```

Go — Optimized

```go
import "sort"

func groupAnagrams(strs []string) [][]string {
	groups := make(map[string][]string)
	for _, s := range strs {
		// Create canonical key
		chars := []byte(s)
		sort.Slice(chars, func(i,j int) bool {
			return chars[i] < chars[j]
		})
		key := string(chars)
		groups[key] = append(groups[key], s)
	}
	result := make([][]string, 0, len(groups))
	for _, g := range groups {
		result = append(result, g)
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n^2 * k)

Space: O(n*k)

OPTIMIZED

Time: O(n*k log k)

Space: O(n*k)

Formula	Memory Tip
Key insight: Two strings are anagrams IFF their sorted versions are equal.	"Sort each word to get its DNA. Same DNA = same family." Sorted string = fingerprint.

Interview Tips & Edge Cases

Alternative key: character frequency array [26]int as string key — avoids sort, O(n*k) time. Ask about Unicode.

---

5 Top K Frequent Elements

Arrays & Hashing

Medium

Problem: Given integer array nums and k, return the k most frequent elements.

Constraints

1 <= nums.length <= 10^5 | k is valid | Answer is unique

Real-World Use Cases

Twitter trending topics, Google autocomplete, log analysis (top error codes), stock market (most traded symbols).

■ What If This Pattern Didn't Exist?

Sort everything by frequency every time: O(n log n). Bucket sort approach: O(n).

■ Memory Analogy

Imagine a ballot box counter. First count all votes (frequency map). Then use buckets where bucket[i] holds all candidates with exactly i votes. Sca

■ Brute Force Approach

Pseudo Code

```
CREATE freq map: num -> count
SORT nums by frequency DESC
RETURN first k unique elements
```

Go — Brute Force

```go
import "sort"

func topKFrequent(nums []int, k int) []int {
	freq := make(map[int]int)
	for _, n := range nums { freq[n]++ }
	keys := make([]int, 0, len(freq))
	for key := range freq { keys = append(keys, key) }
	sort.Slice(keys, func(i,j int) bool {
		return freq[keys[i]] > freq[keys[j]]
	})
	return keys[:k]
}
```

■ Optimized Approach

Pseudo Code

```
// Bucket Sort approach O(n)
BUILD freq map
CREATE buckets[n+1]  // bucket[i] = nums with freq i
FOR num, count in freq:
   buckets[count] = append(buckets[count], num)
SCAN from bucket[n] down to bucket[1]:
   Collect elements until we have k
```

Go — Optimized

```go
func topKFrequent(nums []int, k int) []int {
	freq := make(map[int]int)
	for _, n := range nums { freq[n]++ }
	// bucket[i] holds numbers that appear i times
	buckets := make([][]int, len(nums)+1)
	for num, cnt := range freq {
		buckets[cnt] = append(buckets[cnt], num)
	}
	result := []int{}
	for i := len(buckets) - 1; i >= 0 && len(result) < k; i-- {
		result = append(result, buckets[i]...)
	}
	return result[:k]
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n log n)
Space: O(n)

OPTIMIZED
Time: O(n)
Space: O(n)

Formula	Memory Tip
Bucket index = frequency. Max possible frequency = n. Scan buckets high to low.	"Count votes -> put in frequency buckets -> harvest from top." Bucket = frequency shelf.

Interview Tips & Edge Cases

Heap approach is O(n log k) — better when k << n. Bucket sort is O(n) — better when k ~ n. Know both!

---

6 Product of Array Except Self

Arrays & Hashing

Medium

Problem: Return array where output[i] = product of all elements except nums[i]. No division.

Constraints

2 <= nums.length <= 10^5 | No division allowed | Output[i] != 0

Real-World Use Cases

Signal processing (noise filters), financial risk calculations (exclude one asset), machine learning feature engineering.

What If This Pattern Didn't Exist?

Division approach O(n) but fails when zeros present. Brute force O(n^2) recalculates products for every index.

Memory Analogy

Imagine a relay race. First runner passes the baton left-to-right (prefix product). Then a second runner goes right-to-left (suffix product). Every runn

Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-1:
   product = 1
   FOR j = 0 to n-1:
      IF j != i: product *= nums[j]
   output[i] = product
```

Go — Brute Force

```go
func productExceptSelf(nums []int) []int {
      n := len(nums)
      output := make([]int, n)
      for i := 0; i < n; i++ {
            product := 1
            for j := 0; j < n; j++ {
                  if j != i { product *= nums[j] }
            }
            output[i] = product
      }
      return output
}
```

Optimized Approach

Pseudo Code

```
// Pass 1: prefix products (left to right)
result[0] = 1
FOR i = 1 to n-1: result[i] = result[i-1] * nums[i-1]

// Pass 2: suffix products (right to left)
suffix = 1
FOR i = n-1 down to 0:
   result[i] *= suffix
   suffix *= nums[i]
```

Go — Optimized

```go
func productExceptSelf(nums []int) []int {
	n := len(nums)
	result := make([]int, n)
	// Left pass: prefix products
	result[0] = 1
	for i := 1; i < n; i++ {
		result[i] = result[i-1] * nums[i-1]
	}
	// Right pass: multiply by suffix
	suffix := 1
	for i := n - 1; i >= 0; i-- {
		result[i] *= suffix
		suffix *= nums[i]
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(1) extra (output doesn't count)

Formula	Memory Tip
result[i] = prefix[i] * suffix[i]. Merge in two passes without extra arrays.	"Left relay baton, then right relay baton. Each position = what came before * what comes after."

Interview Tips & Edge Cases

The output array is NOT considered extra space per problem constraints. Zeros: if 2+ zeros, all products = 0. If 1 zero, only that index has non-zer

---

7

Valid Sudoku

Arrays & Hashing

Medium

Problem: Determine if a 9x9 Sudoku board is valid based on current filled cells.

Constraints

Board is 9x9 | Cells contain '1'-'9' or '.'

Real-World Use Cases

Constraint satisfaction systems, puzzle validation engines, form validation with mutual exclusion rules.

What If This Pattern Didn't Exist?

You'd need to check every row, column, and box separately with nested loops — error-prone O(n^3) logic.

■ Memory Analogy

Three security checkpoints: Row police (no duplicates in rows), Column police (no duplicates in cols), Box police (no duplicates in 3x3 boxes). Eac

■ Brute Force Approach

Pseudo Code

```
FOR each row: check no duplicates
FOR each col: check no duplicates
FOR each 3x3 box: check no duplicates
// Using nested loops and sets
```

Go — Brute Force

```go
// Check function helper
func isValidSudoku(board [][]byte) bool {
	// Check rows
	for r := 0; r < 9; r++ {
		seen := make(map[byte]bool)
		for c := 0; c < 9; c++ {
			if board[r][c] == '.' { continue }
			if seen[board[r][c]] { return false }
			seen[board[r][c]] = true
		}
	}
	// Check cols (similar)...
	// Check boxes (similar)...
	return true
}
```

■ Optimized Approach

Pseudo Code

```
USE 3 sets of 9 hash sets each:
   rows[9], cols[9], boxes[9]
FOR each cell (r,c):
   IF cell is '.': skip
   boxIndex = (r/3)*3 + (c/3)
   IF val in rows[r] OR cols[c] OR boxes[boxIdx]:
      RETURN false
   ADD val to all 3 sets
RETURN true
```

Go — Optimized

```go
func isValidSudoku(board [][]byte) bool {
	rows := [9]map[byte]bool{}
	cols := [9]map[byte]bool{}
	boxes := [9]map[byte]bool{}
	for i := range rows {
		rows[i] = make(map[byte]bool)
		cols[i] = make(map[byte]bool)
		boxes[i] = make(map[byte]bool)
	}
	for r := 0; r < 9; r++ {
		for c := 0; c < 9; c++ {
			val := board[r][c]
			if val == '.' { continue }
			box := (r/3)*3 + (c/3)
			if rows[r][val] || cols[c][val] || boxes[box][val] {
				return false
			}
			rows[r][val] = true
			cols[c][val] = true
			boxes[box][val] = true
		}
	}
	return true
}
```

Complexity Analysis

BRUTE FORCE

Time: O(81) = O(1)

Space: O(81) = O(1)

OPTIMIZED

Time: O(1)

Space: O(1) — fixed 9x9 board

Formula	Memory Tip
boxIndex = (row/3)*3 + (col/3). This maps each 3x3 box to 0-8.	"Box formula: (r/3)*3 + (c/3)". This is THE key formula. Everything else is just set membership.

Interview Tips & Edge Cases

All complexities are O(1) since board is always 9x9. Focus on the box index formula in interview. Also validate: not solving, just validating.

---

8 Encode and Decode Strings

Arrays & Hashing

Medium

Problem: Design encode/decode functions to serialize a list of strings to a single string and back.

Constraints

Strings can contain any character including delimiters

■ Real-World Use Cases

Network serialization (HTTP chunked encoding), message queues (Kafka), RPC protocols, file storage systems.

■ What If This Pattern Didn't Exist?

Simple delimiters like '|' fail if strings contain '|'. Without length-prefix encoding, you can't safely reconstruct the original list.

■ Memory Analogy

Like packing boxes for shipping. Instead of just taping them together (risky — what if a box has tape on it?), you write '12#' (the size of the next ite

■ Brute Force Approach

Pseudo Code

```
// Naive: join with special delimiter
ENCODE: join(strs, chr(0))  // null byte
DECODE: split(encoded, chr(0))
// FAILS: if strings contain null bytes
```

Go — Brute Force

```go
// Naive approach (fails with special chars)
func encode(strs []string) string {
      return strings.Join(strs, "|")  // FRAGILE
}
func decode(s string) []string {
      return strings.Split(s, "|")  // WRONG if | in string
}
```

■ Optimized Approach

Pseudo Code

```
ENCODE:
   FOR each str: output += len(str) + '#' + str

DECODE:
   i = 0
   WHILE i < len(encoded):
      j = find '#' starting at i
      length = parseInt(encoded[i:j])
      i = j + 1
      result.append(encoded[i : i+length])
      i += length
```

Go — Optimized

```go
import (
	"fmt"
	"strconv"
	"strings"
)

func encode(strs []string) string {
	var sb strings.Builder
	for _, s := range strs {
		sb.WriteString(fmt.Sprintf("%d#%s", len(s), s))
	}
	return sb.String()
}

func decode(s string) []string {
	result := []string{}
	i := 0
	for i < len(s) {
		j := i
		for s[j] != '#' { j++ }
		length, _ := strconv.Atoi(s[i:j])
		i = j + 1
		result = append(result, s[i:i+length])
		i += length
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n*k)
Space: O(n*k)

OPTIMIZED
Time: O(n*k)
Space: O(n*k)

Formula	Memory Tip
length-prefix: encode = len(s) + '#' + s. Decode: read until '#', parse length, slice next N chars.	"Size tag before each item — like luggage weight labels. Read the weight, then grab exactly that much."

Interview Tips & Edge Cases

Key insight is delimiter-free encoding. Why not escape? More complex. Length prefix is clean, O(1) per char. This pattern is used in HTTP chunke

---

9 Longest Consecutive Sequence

Arrays & Hashing

Medium

Problem: Given unsorted array, find length of longest sequence of consecutive integers.

■ Constraints

0 <= nums.length <= 10^5 | Must be O(n) time

■ Real-World Use Cases

Finding gaps in time-series data, ID range analysis, detecting missing records in databases.

■ What If This Pattern Didn't Exist?

Sorting gives O(n log n). Without hash sets, checking 'does n+1 exist?' is O(n) per element = O(n^2) total.

■ Memory Analogy

Imagine a row of numbered houses. A sequence starts only at houses that have NO left neighbor (no house n-1). From there, count how far right y

■ Brute Force Approach

Pseudo Code

```
SORT array
current_streak = 1, max_streak = 1
FOR i = 1 to n-1:
   IF nums[i] == nums[i-1]+1: streak++
   ELSE IF nums[i] != nums[i-1]: streak = 1
   max_streak = max(max_streak, streak)
```

Go — Brute Force

```go
import "sort"

func longestConsecutive(nums []int) int {
      if len(nums) == 0 { return 0 }
      sort.Ints(nums)
      maxLen, cur := 1, 1
      for i := 1; i < len(nums); i++ {
            if nums[i] == nums[i-1]+1 { cur++
            } else if nums[i] != nums[i-1] { cur = 1 }
            if cur > maxLen { maxLen = cur }
      }
      return maxLen
}
```

■ Optimized Approach

Pseudo Code

```
ADD all nums to hash set
maxLen = 0
FOR each num in set:
   IF num-1 NOT in set:  // sequence start!
      length = 1
      WHILE num+length in set: length++
      maxLen = max(maxLen, length)
RETURN maxLen
```

Go — Optimized

```go
func longestConsecutive(nums []int) int {
	numSet := make(map[int]bool)
	for _, n := range nums { numSet[n] = true }
	maxLen := 0
	for num := range numSet {
		// Only start counting from sequence beginnings
		if !numSet[num-1] {
			length := 1
			for numSet[num+length] { length++ }
			if length > maxLen { maxLen = length }
		}
	}
	return maxLen
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n log n)

Space: O(1)

OPTIMIZED

Time: O(n)

Space: O(n)

Formula	Memory Tip
Start sequence ONLY when num-1 doesn't exist. Total work across all starts = O(n).	"No left neighbor = sequence captain. Count rightward from captains only."

Interview Tips & Edge Cases

Why O(n)? Each number is visited at most twice (once as start check, once as extension). The 'no left neighbor' guard ensures no redundant starts

■ Two Pointers

10

Valid Palindrome

Two Pointers

Easy

Problem: Given string s, return true if it is a palindrome (considering only alphanumeric, ignoring case).

■ Constraints

1 <= s.length <= 2*10^5 | Contains ASCII chars

■ Real-World Use Cases

DNA sequence validation, text preprocessing in NLP, URL/username validation, input sanitization.

■ What If This Pattern Didn't Exist?

You'd clean the string into a new string and reverse-compare. Extra O(n) space. Two pointers: O(1) space.

■ Memory Analogy

Two people walking toward each other from opposite ends of a bridge, checking if the tiles match. If they ever see different tiles — not a palindrom

■ Brute Force Approach

Pseudo Code

```
CLEAN string: keep only alphanumeric, lowercase
RETURN cleaned == reverse(cleaned)
```

Go — Brute Force

```go
import (
	"strings"
	"unicode"
)
func isPalindrome(s string) bool {
	var clean strings.Builder
	for _, c := range s {
		if unicode.IsLetter(c) || unicode.IsDigit(c) {
			clean.WriteRune(unicode.ToLower(c))
		}
	}
	cs := clean.String()
	for i, j := 0, len(cs)-1; i < j; i, j = i+1, j-1 {
		if cs[i] != cs[j] { return false }
	}
	return true
}
```

■ Optimized Approach

Pseudo Code

```
left = 0, right = len(s)-1
WHILE left < right:
   SKIP non-alphanumeric from left
   SKIP non-alphanumeric from right
   IF lower(s[left]) != lower(s[right]): RETURN false
   left++, right--
RETURN true
```

Go — Optimized

```go
func isPalindrome(s string) bool {
	isAlNum := func(c byte) bool {
		return (c>='a'&&c<='z')||(c>='A'&&c<='Z')||(c>='0'&&c<='9')
	}
	toLow := func(c byte) byte {
		if c>='A'&&c<='Z' { return c+32 }; return c
	}
	l, r := 0, len(s)-1
	for l < r {
		for l < r && !isAlNum(s[l]) { l++ }
		for l < r && !isAlNum(s[r]) { r-- }
		if toLow(s[l]) != toLow(s[r]) { return false }
		l++; r--
	}
	return true
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n)
Space: O(n)

OPTIMIZED
Time: O(n)
Space: O(1)

Formula	Memory Tip
Two pointers converging. Skip non-alphanumeric. Compare case-insensitively.	"Two people walking toward each other. Skip the potholes (non-alnum). Meet in the middle = palindrome."

Interview Tips & Edge Cases

Optimization is about SPACE here, not time. Both are O(n). The in-place two pointer saves the O(n) string allocation.

---

11 Two Sum II - Input Array Is Sorted

Medium

Two Pointers

Problem: Given sorted array, find two numbers that add up to target. Return 1-indexed positions.

Constraints

2 <= numbers.length <= 3*10^4 | Sorted in non-decreasing order | Exactly one solution

Real-World Use Cases

Database sorted index lookups, merge operations in external sorting, financial target matching on sorted data.

What If This Pattern Didn't Exist?

Hash map works but ignores the sorted property — wastes the O(n log n) pre-sorting investment. Two pointers: O(n) and O(1) space.

Memory Analogy

A number line with two fingers. If their sum is too big, move the right finger left (reduce). If too small, move the left finger right (increase). The sorte

Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-1:
   FOR j = i+1 to n-1:
      IF numbers[i] + numbers[j] == target:
         RETURN [i+1, j+1]
```

Go — Brute Force

```go
func twoSumII(numbers []int, target int) []int {
      for i := 0; i < len(numbers); i++ {
            for j := i + 1; j < len(numbers); j++ {
                  if numbers[i]+numbers[j] == target {
                        return []int{i+1, j+1}
                  }
            }
      }
      return nil
}
```

Optimized Approach

Pseudo Code

```
left = 0, right = n-1
WHILE left < right:
   sum = numbers[left] + numbers[right]
   IF sum == target: RETURN [left+1, right+1]
   IF sum < target: left++   // need bigger
   ELSE: right--             // need smaller
```

Go — Optimized

```go
func twoSumII(numbers []int, target int) []int {
	l, r := 0, len(numbers)-1
	for l < r {
		sum := numbers[l] + numbers[r]
		switch {
		case sum == target:
			return []int{l+1, r+1}
		case sum < target:
			l++
		default:
			r--
		}
	}
	return nil
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(1)

Formula	Memory Tip
sum < target → left++ (increase sum). sum > target → right-- (decrease sum).	"Too small? Stretch left pointer right. Too big? Pull right pointer left. Like adjusting a rubber band."

Interview Tips & Edge Cases

Works ONLY on sorted arrays. This is the pattern used inside 3Sum's inner loop. Must return 1-indexed result.

---

12 3Sum

Two Pointers

Medium

Problem: Find all unique triplets [a,b,c] such that a+b+c=0.

Constraints

3 <= nums.length <= 3000 | No duplicate triplets in result

Real-World Use Cases

3D coordinate systems, finding complementary financial positions, physics collision detection.

What If This Pattern Didn't Exist?

Pure brute force: O(n^3) — 27 billion ops for n=3000. Two-pointer inner loop brings this to O(n^2).

Memory Analogy

Fix one person (i), then use the 'rubber band' two-pointer trick for the remaining sorted array. The fixed person 'anchors' the problem. Skip duplicat

Brute Force Approach

Pseudo Code

```
SORT nums
FOR i: FOR j>i: FOR k>j:
   IF nums[i]+nums[j]+nums[k]==0:
      ADD {i,j,k} to set (avoid dups)
```

Go — Brute Force

```go
// O(n^3) with dedup via set
func threeSum(nums []int) [][]int {
      sort.Ints(nums)
      seen := map[[3]int]bool{}
      res := [][]int{}
      for i := 0; i < len(nums)-2; i++ {
            for j := i+1; j < len(nums)-1; j++ {
                  for k := j+1; k < len(nums); k++ {
                        if nums[i]+nums[j]+nums[k]==0 {
                              key := [3]int{nums[i],nums[j],nums[k]}
                              if !seen[key] { res=append(res,[]int{nums[i],nums[j],nums[k]}); seen[key]=true }
                        }
                  }
            }
      }
      return res
}
```

Optimized Approach

Pseudo Code

```
SORT nums
FOR i = 0 to n-3:
   SKIP duplicates at i
   left = i+1, right = n-1
   WHILE left < right:
      sum = nums[i]+nums[left]+nums[right]
      IF sum==0: add triplet, skip dups, l++, r--
      IF sum<0: left++
      IF sum>0: right--
```

Go — Optimized

```go
import "sort"

func threeSum(nums []int) [][]int {
	sort.Ints(nums)
	result := [][]int{}
	for i := 0; i < len(nums)-2; i++ {
		if i > 0 && nums[i] == nums[i-1] { continue }
		l, r := i+1, len(nums)-1
		for l < r {
			sum := nums[i]+nums[l]+nums[r]
			if sum == 0 {
				result = append(result, []int{nums[i],nums[l],nums[r]})
				for l<r && nums[l]==nums[l+1] { l++ }
				for l<r && nums[r]==nums[r-1] { r-- }
				l++; r--
			} else if sum < 0 { l++
			} else { r-- }
		}
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^3)
Space: O(n)

OPTIMIZED
Time: O(n^2)
Space: O(1) extra

Formula	Memory Tip
Fix i. Two pointers l=i+1, r=n-1. Skip duplicates at all 3 levels.	"Sort, fix one anchor, two-pointer the rest. Skip duplicates at EVERY level."

Interview Tips & Edge Cases

Early exit: if nums[i] > 0, break (sorted, so no triplet can sum to 0). Dedup is the tricky part — skip after finding a valid triplet AND skip anchors.

---

13 Container With Most Water

Two Pointers

Medium

Problem: Given height array, find two lines that together with x-axis forms max water container.

Constraints

2 <= height.length <= 10^5 | 0 <= height[i] <= 10^4

Real-World Use Cases

Reservoir capacity planning, maximizing storage efficiency, bandwidth optimization between two endpoints.

■ What If This Pattern Didn't Exist?

Check all pairs: O(n^2). Two pointers: O(n) by greedily moving the shorter wall inward.

■ Memory Analogy

Two walls forming a bucket. Water = min(left wall, right wall) * distance. To maximize, start with widest bucket (full width). Then try to improve by m

■ Brute Force Approach

Pseudo Code

```
maxWater = 0
FOR i = 0 to n-1:
   FOR j = i+1 to n-1:
      water = min(height[i], height[j]) * (j-i)
      maxWater = max(maxWater, water)
```

Go — Brute Force

```go
func maxArea(height []int) int {
	maxW := 0
	for i := 0; i < len(height); i++ {
		for j := i+1; j < len(height); j++ {
			h := height[i]
			if height[j] < h { h = height[j] }
			if w := h * (j - i); w > maxW { maxW = w }
		}
	}
	return maxW
}
```

■ Optimized Approach

Pseudo Code

```
left = 0, right = n-1, maxWater = 0
WHILE left < right:
   water = min(height[left], height[right]) * (right - left)
   maxWater = max(maxWater, water)
   IF height[left] < height[right]: left++
   ELSE: right--
```

Go — Optimized

```go
func maxArea(height []int) int {
	l, r, maxW := 0, len(height)-1, 0
	for l < r {
		h := height[l]
		if height[r] < h { h = height[r] }
		if area := h * (r - l); area > maxW { maxW = area }
		if height[l] < height[r] { l++ } else { r-- }
	}
	return maxW
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(1)

Formula	Memory Tip
Area = min(h[l], h[r]) * (r-l). Move the pointer with the SMALLER height.	"Move the SHORT wall. The tall wall is already doing its best — moving it can only hurt." Greedy correctness proof: moving the taller wall can never increase area.

Interview Tips & Edge Cases

The greedy argument: current area is limited by the shorter wall. Keeping it and shrinking width always gives <= current. So move the shorter one

---

14 Trapping Rain Water

Two Pointers

Hard

Problem: Given elevation map, compute how much water it can trap after raining.

Constraints

1 <= height.length <= 2*10^4 | 0 <= height[i] <= 10^5

Real-World Use Cases

Urban flood modeling, terrain analysis, histogram-based storage problems, CPU core utilization visualization.

What If This Pattern Didn't Exist?

Computing max left/right for each position separately: O(n) space + O(n) time. Two pointers: O(1) space.

Memory Analogy

Water at each bar = min(tallest bar to its left, tallest bar to its right) - bar's own height. Two pointers carry the max seen from each side, meeting in

Brute Force Approach

Pseudo Code

```
FOR each position i:
   leftMax = max(height[0..i])
   rightMax = max(height[i..n-1])
   water[i] = max(0, min(leftMax,rightMax) - height[i])
RETURN sum(water)
```

Go — Brute Force

```go
func trap(height []int) int {
      n := len(height)
      leftMax := make([]int, n)
      rightMax := make([]int, n)
      leftMax[0] = height[0]
      for i := 1; i < n; i++ {
            if height[i]>leftMax[i-1] { leftMax[i]=height[i] } else { leftMax[i]=leftMax[i-1] }
      }
      rightMax[n-1] = height[n-1]
      for i := n-2; i >= 0; i-- {
            if height[i]>rightMax[i+1] { rightMax[i]=height[i] } else { rightMax[i]=rightMax[i+1] }
      }
      total := 0
      for i := 0; i < n; i++ {
            m := leftMax[i]; if rightMax[i]<m { m=rightMax[i] }
            if w := m - height[i]; w > 0 { total += w }
      }
      return total
}
```

Optimized Approach

Pseudo Code

```
left=0, right=n-1, leftMax=0, rightMax=0, water=0
WHILE left < right:
   IF height[left] <= height[right]:
      IF height[left] >= leftMax: leftMax = height[left]
      ELSE: water += leftMax - height[left]
      left++
   ELSE:
      IF height[right] >= rightMax: rightMax = height[right]
      ELSE: water += rightMax - height[right]
      right--
```

Go — Optimized

```go
func trap(height []int) int {
	l, r := 0, len(height)-1
	leftMax, rightMax := 0, 0
	water := 0
	for l < r {
		if height[l] <= height[r] {
			if height[l] >= leftMax {
				leftMax = height[l]
			} else {
				water += leftMax - height[l]
			}
			l++
		} else {
			if height[r] >= rightMax {
				rightMax = height[r]
			} else {
				water += rightMax - height[r]
			}
			r--
		}
	}
	return water
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n)

Space: O(n)

OPTIMIZED

Time: O(n)

Space: O(1)

Formula	Memory Tip
water[i] = max(0, min(leftMax, rightMax) - height[i]). Process side with smaller max first.	"Water fills to the level of the SHORTER surrounding wall. Move from the side with the smaller max — it's the bottleneck."

Interview Tips & Edge Cases

Two pointer space optimization is key here. The brute force is actually already O(n) time. Interview wants the O(1) space solution. Stack-based O(

■ Sliding Window

15

Best Time to Buy and Sell Stock

Sliding Window

Easy

Problem: Find maximum profit from one buy and one sell transaction.

■ Constraints

1 <= prices.length <= 10^5 | 0 <= prices[i] <= 10^4 | Buy before sell

■ Real-World Use Cases

Stock trading algorithms, cryptocurrency arbitrage, resource procurement optimization.

■ What If This Pattern Didn't Exist?

Check all pairs (buy day, sell day): O(n^2). Sliding window / Kadane's-like: O(n).

■ Memory Analogy

Imagine driving from city to city looking for the cheapest gas (buy) and the most expensive selling point (sell) ahead. You track the cheapest gas s

■ Brute Force Approach

Pseudo Code

```
maxProfit = 0
FOR i = 0 to n-1: (buy day)
   FOR j = i+1 to n-1: (sell day)
      profit = prices[j] - prices[i]
      maxProfit = max(maxProfit, profit)
```

Go — Brute Force

```go
func maxProfit(prices []int) int {
      maxP := 0
      for i := 0; i < len(prices); i++ {
            for j := i+1; j < len(prices); j++ {
                  if p := prices[j]-prices[i]; p > maxP { maxP = p }
            }
      }
      return maxP
}
```

■ Optimized Approach

Pseudo Code

```
minPrice = prices[0], maxProfit = 0
FOR each price:
   IF price < minPrice: update minPrice
   ELSE: maxProfit = max(maxProfit, price - minPrice)
```

Go — Optimized

```go
func maxProfit(prices []int) int {
	minPrice := prices[0]
	maxP := 0
	for _, price := range prices {
		if price < minPrice {
			minPrice = price
		} else if profit := price - minPrice; profit > maxP {
			maxP = profit
		}
	}
	return maxP
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(1)

Formula	Memory Tip
profit = currentPrice - minPriceSoFar. Update minPrice greedily.	"Track the valley (min). Each mountain asks: if I sold here, what's the profit from the deepest valley?"

Interview Tips & Edge Cases

Two windows: buy pointer (min so far) and sell pointer (current). This is effectively a 1-variable sliding window minimum. Follow-up: multiple transa

---

16

Longest Substring Without Repeating Characters

Medium

Sliding Window

Problem: Find length of longest substring without any repeating characters.

Constraints

0 <= s.length <= 5*10^4 | English letters, digits, symbols, spaces

Real-World Use Cases

Session token generation (unique chars), password strength checking, stream processing (unique data windows).

What If This Pattern Didn't Exist?

Checking every substring: O(n^3) or O(n^2). Sliding window: O(n) by never re-checking characters.

Memory Analogy

A sliding spotlight on a stage. When a new actor enters who's already on stage, the spotlight's left edge moves right until that actor exits. The spot

Brute Force Approach

Pseudo Code

```
maxLen = 0
FOR i = 0 to n-1:
   FOR j = i to n-1:
      IF s[i..j] has all unique chars:
         maxLen = max(maxLen, j-i+1)
      ELSE: break
```

Go — Brute Force

```go
func lengthOfLongestSubstring(s string) int {
	maxLen := 0
	for i := 0; i < len(s); i++ {
		seen := make(map[byte]bool)
		for j := i; j < len(s); j++ {
			if seen[s[j]] { break }
			seen[s[j]] = true
			if j-i+1 > maxLen { maxLen = j-i+1 }
		}
	}
	return maxLen
}
```

Optimized Approach

Pseudo Code

```
charIndex map, left = 0, maxLen = 0
FOR right = 0 to n-1:
   IF s[right] in map AND map[s[right]] >= left:
      left = map[s[right]] + 1  // jump left past duplicate
   map[s[right]] = right
   maxLen = max(maxLen, right - left + 1)
```

Go — Optimized

```go
func lengthOfLongestSubstring(s string) int {
	charIdx := make(map[byte]int)
	left, maxLen := 0, 0
	for right := 0; right < len(s); right++ {
		if idx, ok := charIdx[s[right]]; ok && idx >= left {
			left = idx + 1 // shrink window past duplicate
		}
		charIdx[s[right]] = right
		if w := right - left + 1; w > maxLen { maxLen = w }
	}
	return maxLen
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2)
Space: O(min(n,charset))

OPTIMIZED
Time: O(n)
Space: O(min(n,charset))

Formula	Memory Tip
Window size = right - left + 1. On duplicate: left = last_seen_index + 1.	"Map stores last seen position. Duplicate found? Jump left pointer PAST its last position."

Interview Tips & Edge Cases

The check `idx >= left` is crucial — ensures we don't jump left backward. For ASCII-only strings, use [128]int array instead of map for faster consta

---

17 Longest Repeating Character Replacement

Medium

Sliding Window

Problem: Replace at most k characters to make longest substring of all same characters. Return that length.

Constraints

1 <= s.length <= 10^5 | s has uppercase letters | 0 <= k <= s.length

Real-World Use Cases

DNA mutation analysis (minimum edits), network packet correction, string normalization.

What If This Pattern Didn't Exist?

Try every window and every target character: O(n^2 * 26). Sliding window: O(n).

Memory Analogy

A paint roller covering a wall. Track the most common color in the current window. If (window size - max frequency) > k, you'd need too many pain

Brute Force Approach

Pseudo Code

```
maxLen = 0
FOR each char c in 'A'-'Z':
   FOR i = 0 to n-1:
      count = 0, replacements = 0
      FOR j = i to n-1:
         IF s[j] != c: replacements++
         IF replacements > k: break
         maxLen = max(maxLen, j-i+1)
```

Go — Brute Force

```go
func characterReplacement(s string, k int) int {
	maxLen := 0
	for c := byte('A'); c <= 'Z'; c++ {
		left := 0
		replacements := 0
		for right := 0; right < len(s); right++ {
			if s[right] != c { replacements++ }
			for replacements > k {
				if s[left] != c { replacements-- }
				left++
			}
			if w := right-left+1; w > maxLen { maxLen = w }
		}
	}
	return maxLen
}
```

Optimized Approach

Pseudo Code

```
freq[26], left=0, maxFreq=0, maxLen=0
FOR right = 0 to n-1:
   freq[s[right]-'A']++
   maxFreq = max(maxFreq, freq[s[right]-'A'])
   // Window invalid if: size - maxFreq > k
   IF (right - left + 1) - maxFreq > k:
      freq[s[left]-'A']--
      left++
   maxLen = max(maxLen, right-left+1)
```

Go — Optimized

```go
func characterReplacement(s string, k int) int {
	freq := [26]int{}
	left, maxFreq, maxLen := 0, 0, 0
	for right := 0; right < len(s); right++ {
		freq[s[right]-'A']++
		if f := freq[s[right]-'A']; f > maxFreq { maxFreq = f }
		// replacements needed = window_size - maxFreq
		if (right-left+1)-maxFreq > k {
			freq[s[left]-'A']--
			left++
		}
		if w := right-left+1; w > maxLen { maxLen = w }
	}
	return maxLen
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n*26)
Space: O(1)

OPTIMIZED
Time: O(n)
Space: O(1)

Formula	Memory Tip
replacements_needed = window_size - max_frequency. If > k, shrink window.	"Count chars in window. Max freq char stays. Replace the rest. If replacements > k, shrink."

Interview Tips & Edge Cases

maxFreq never decreases — this is the key insight. The window size is monotonically non-decreasing, so we only need to move left by 1 when inv

---

18 Permutation in String

Sliding Window

Medium

Problem: Return true if s2 contains a permutation of s1 as a substring.

Constraints

1 <= s1.length, s2.length <= 10^4 | lowercase letters only

Real-World Use Cases

Anagram search in text, plagiarism detection, bioinformatics pattern matching.

What If This Pattern Didn't Exist?

Generate all permutations of s1: O(n! * m). Fixed-size window: O(n+m).

Memory Analogy

Slide a window of length len(s1) across s2. At each position, check if the window contains exactly the same character frequencies as s1. Instead c

Brute Force Approach

Pseudo Code

```
need = freq map of s1
FOR i = 0 to len(s2)-len(s1):
   window = s2[i : i+len(s1)]
   IF freq(window) == need: RETURN true
RETURN false
```

Go — Brute Force

```go
func checkInclusion(s1 string, s2 string) bool {
	if len(s1) > len(s2) { return false }
	need := [26]int{}
	for _, c := range s1 { need[c-'a']++ }
	for i := 0; i <= len(s2)-len(s1); i++ {
		window := [26]int{}
		for _, c := range s2[i:i+len(s1)] { window[c-'a']++ }
		if window == need { return true }
	}
	return false
}
```

Optimized Approach

Pseudo Code

```
need = freq(s1), window = freq(s2[:len(s1)-1])
matches = count of chars where need[c] == window[c]
FOR right = len(s1)-1 to len(s2)-1:
   ADD s2[right] to window, update matches
   IF matches == 26: RETURN true
   REMOVE s2[right-len(s1)+1] from window, update matches
```

Go — Optimized

```go
func checkInclusion(s1 string, s2 string) bool {
	if len(s1) > len(s2) { return false }
	need, window := [26]int{}, [26]int{}
	for i := range s1 { need[s1[i]-'a']++; window[s2[i]-'a']++ }
	matches := 0
	for i := 0; i < 26; i++ { if need[i]==window[i] { matches++ } }
	for r := len(s1); r < len(s2); r++ {
		l := r - len(s1)
		if matches == 26 { return true }
		// Add right char
		in := s2[r]-'a'
		window[in]++
		if need[in]==window[in] { matches++ } else if need[in]+1==window[in] { matches-- }
		// Remove left char
		out := s2[l]-'a'
		if need[out]==window[out] { matches-- } else if need[out]-1==window[out] { matches++ }
		window[out]--
	}
	return matches == 26
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n*m)
Space: O(1)

OPTIMIZED
Time: O(n+m)
Space: O(1)

Formula	Memory Tip
Fixed window size = len(s1). Track 'matches' (count of chars with equal freq). matches==26 means permutation found.	"26 judges, each watching one letter. All 26 give thumbs up = permutation found." matches counter tracks agreement.

Interview Tips & Edge Cases

The 'matches' optimization avoids comparing all 26 chars each step. Key: when a char's count crosses need[c], either increment or decrement ma

---

19 Minimum Window Substring

Sliding Window

Hard

Problem: Find minimum window in s that contains all characters of t.

Constraints

1 <= s.length <= 10^5 | 1 <= t.length <= 10^5 | May have duplicate chars in t

Real-World Use Cases

Search snippet generation, keyword highlighting in documents, biological sequence analysis.

What If This Pattern Didn't Exist?

Checking all substrings: O(n^2 * m). Variable sliding window: O(n+m).

Memory Analogy

You need to collect all ingredients in a treasure hunt (t = list of ingredients). Expand your search area right until you have everything. Then shrink f

Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-1:
   FOR j = i to n-1:
      IF s[i..j] contains all chars of t:
         track minimum length window
```

Go — Brute Force

```go
func minWindow(s string, t string) string {
	best := ""
	need := map[byte]int{}
	for i := range t { need[t[i]]++ }
	for i := 0; i < len(s); i++ {
		window := map[byte]int{}
		for j := i; j < len(s); j++ {
			window[s[j]]++
			valid := true
			for c, cnt := range need {
				if window[c] < cnt { valid = false; break }
			}
			if valid && (best=="" || j-i+1 < len(best)) { best = s[i:j+1] }
		}
	}
	return best
}
```

Optimized Approach

Pseudo Code

```
need = freq(t), formed = 0, required = len(unique chars in t)
window map, left = 0
FOR right = 0 to n-1:
   ADD s[right] to window
   IF window[s[right]] == need[s[right]]: formed++
   WHILE formed == required:
      UPDATE answer if smaller
      REMOVE s[left] from window
      IF window[s[left]] < need[s[left]]: formed--
      left++
```

Go — Optimized

```go
func minWindow(s string, t string) string {
	need := map[byte]int{}
	for i := range t { need[t[i]]++ }
	window := map[byte]int{}
	formed, required := 0, len(need)
	l, resL, resR := 0, -1, -1
	for r := 0; r < len(s); r++ {
		window[s[r]]++
		if cnt, ok := need[s[r]]; ok && window[s[r]]==cnt { formed++ }
		for formed == required {
			if resL == -1 || r-l < resR-resL { resL, resR = l, r }
			window[s[l]]--
			if cnt, ok := need[s[l]]; ok && window[s[l]] < cnt { formed-- }
			l++
		}
	}
	if resL == -1 { return "" }
	return s[resL:resR+1]
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n^2 * m)
Space: O(m)

OPTIMIZED
Time: O(n+m)
Space: O(m)

Formula	Memory Tip
Expand right until all chars satisfied. Shrink left while still satisfied. formed==required is the check.	"Expand until full, shrink until broken, track minimum. Caterpillar motion on the string."

Interview Tips & Edge Cases

'formed' tracks distinct chars with sufficient frequency. 'required' is the total distinct chars in t. This avoids checking the entire window at each step.

---

20 Sliding Window Maximum

Sliding Window

Hard

Problem: Return array of maximums of each sliding window of size k.

Constraints

1 <= nums.length <= 10^5 | -10^4 <= nums[i] <= 10^4 | 1 <= k <= nums.length

Real-World Use Cases

Real-time analytics (peak CPU usage), sensor data processing, financial technical indicators (moving max).

■ What If This Pattern Didn't Exist?

For each window, scan to find max: O(n*k). Deque (monotonic queue): O(n).

■ Memory Analogy

A VIP bouncer at a concert who maintains a 'most important person' list. Newcomers who are more important kick out everyone less important beh

■ Brute Force Approach

Pseudo Code

```
FOR i = 0 to n-k:
   max = nums[i]
   FOR j = i+1 to i+k-1:
      max = max(max, nums[j])
   result.append(max)
```

Go — Brute Force

```go
func maxSlidingWindow(nums []int, k int) []int {
	result := []int{}
	for i := 0; i <= len(nums)-k; i++ {
		maxVal := nums[i]
		for j := i+1; j < i+k; j++ {
			if nums[j] > maxVal { maxVal = nums[j] }
		}
		result = append(result, maxVal)
	}
	return result
}
```

■ Optimized Approach

Pseudo Code

```
deque stores indices (decreasing values)
FOR i = 0 to n-1:
   REMOVE from front if out of window (idx <= i-k)
   REMOVE from back while nums[back] <= nums[i]
   PUSH i to back
   IF i >= k-1: result.append(nums[deque.front])
```

Go — Optimized

```go
func maxSlidingWindow(nums []int, k int) []int {
	deque := []int{} // stores indices
	result := []int{}
	for i := 0; i < len(nums); i++ {
		// Remove out-of-window elements
		for len(deque) > 0 && deque[0] < i-k+1 {
			deque = deque[1:]
		}
		// Maintain decreasing order
		for len(deque) > 0 && nums[deque[len(deque)-1]] < nums[i] {
			deque = deque[:len(deque)-1]
		}
		deque = append(deque, i)
		if i >= k-1 {
			result = append(result, nums[deque[0]])
		}
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n*k)

Space: O(1)

OPTIMIZED

Time: O(n)

Space: O(k)

Formula	Memory Tip
Monotonic deque: front = max of window. Remove rear when new element is bigger. Remove front when outside window.	"VIP list: newcomer kicks out smaller VIPs behind them. Old VIPs outside the window get removed from front."

Interview Tips & Edge Cases

Deque stores INDICES not values. Two invariants: 1) Front is always window max. 2) Values from front to back are decreasing. Each element is a

■ Math & Geometry

21 Rotate Image

Math & Geometry

Medium

Problem: Rotate an n x n matrix 90 degrees clockwise in-place.

■ Constraints

n == matrix.length == matrix[i].length | 1 <= n <= 20 | In-place rotation

■ Real-World Use Cases

Image processing (photo rotation), game board transformations, screen orientation changes.

■ What If This Pattern Didn't Exist?

Creating a new matrix: O(n^2) space. In-place: O(1) space with the transpose+reverse trick.

■ Memory Analogy

Two dance moves: First, flip the matrix across its main diagonal (transpose). Then, flip each row horizontally (reverse rows). These two flips comb

■ Brute Force Approach

Pseudo Code

```
CREATE new n x n matrix
FOR i = 0 to n-1:
   FOR j = 0 to n-1:
      new_matrix[j][n-1-i] = matrix[i][j]
COPY new_matrix back to matrix
```

Go — Brute Force

```go
func rotate(matrix [][]int) {
      n := len(matrix)
      tmp := make([][]int, n)
      for i := range tmp { tmp[i] = make([]int, n) }
      for i := 0; i < n; i++ {
            for j := 0; j < n; j++ {
                  tmp[j][n-1-i] = matrix[i][j]
            }
      }
      for i := range matrix { copy(matrix[i], tmp[i]) }
}
```

■ Optimized Approach

Pseudo Code

```
// Step 1: Transpose (swap across diagonal)
FOR i = 0 to n-1:
   FOR j = i+1 to n-1:
      SWAP matrix[i][j] and matrix[j][i]

// Step 2: Reverse each row
FOR each row:
   REVERSE row
```

Go — Optimized

```go
func rotate(matrix [][]int) {
	n := len(matrix)
	// Step 1: Transpose
	for i := 0; i < n; i++ {
		for j := i + 1; j < n; j++ {
			matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
		}
	}
	// Step 2: Reverse each row
	for i := 0; i < n; i++ {
		for l, r := 0, n-1; l < r; l, r = l+1, r-1 {
			matrix[i][l], matrix[i][r] = matrix[i][r], matrix[i][l]
		}
	}
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n^2)

Space: O(n^2)

OPTIMIZED

Time: O(n^2)

Space: O(1)

Formula	Memory Tip
90deg clockwise = Transpose + Reverse each row. Counter-clockwise = Reverse each row + Transpose.	"Clockwise: T then R (Transpose then Reverse rows). Counter-clockwise: R then T."

Interview Tips & Edge Cases

The mathematical formula for 90deg CW rotation: new[j][n-1-i] = old[i][j]. But in-place doesn't use this directly. Counter-clockwise = reverse each c

---

22 Spiral Matrix

Math & Geometry

Medium

Problem: Return all elements of the matrix in spiral order.

Constraints

m == matrix.length | n == matrix[i].length | 1 <= m,n <= 10

Real-World Use Cases

Printer path generation, CNC machine toolpaths, data serialization for 2D structures, image scanning patterns.

What If This Pattern Didn't Exist?

Recursive boundary tracking or complex index math. The 4-boundary approach is clean and avoids index errors.

Memory Analogy

Peeling an onion layer by layer. Each pass you peel the outermost ring: go right along top, go down along right, go left along bottom, go up along l

Brute Force Approach

Pseudo Code

```
// Direction simulation with visited array
directions = [(0,1),(1,0),(0,-1),(-1,0)]
visited = m x n boolean array
WALK with direction changes
```

Go — Brute Force

```go
func spiralOrder(matrix [][]int) []int {
	m, n := len(matrix), len(matrix[0])
	visited := make([][]bool, m)
	for i := range visited { visited[i] = make([]bool, n) }
	dirs := [][2]int{{0,1},{1,0},{0,-1},{-1,0}}
	r, c, d := 0, 0, 0
	result := make([]int, m*n)
	for i := range result {
		result[i] = matrix[r][c]
		visited[r][c] = true
		nr, nc := r+dirs[d][0], c+dirs[d][1]
		if nr<0||nr>=m||nc<0||nc>=n||visited[nr][nc] { d=(d+1)%4; nr,nc=r+dirs[d][0],c+dirs[d][1] }
		r, c = nr, nc
	}
	return result
}
```

Optimized Approach

Pseudo Code

```
top=0, bottom=m-1, left=0, right=n-1
WHILE top <= bottom AND left <= right:
   Traverse top row left->right; top++
   Traverse right col top->bottom; right--
   IF top<=bottom: Traverse bottom row right->left; bottom--
   IF left<=right: Traverse left col bottom->top; left++
```

Go — Optimized

```go
func spiralOrder(matrix [][]int) []int {
	top, bottom := 0, len(matrix)-1
	left, right := 0, len(matrix[0])-1
	result := []int{}
	for top <= bottom && left <= right {
		for c := left; c <= right; c++ { result = append(result, matrix[top][c]) }
		top++
		for r := top; r <= bottom; r++ { result = append(result, matrix[r][right]) }
		right--
		if top <= bottom {
			for c := right; c >= left; c-- { result = append(result, matrix[bottom][c]) }
			bottom--
		}
		if left <= right {
			for r := bottom; r >= top; r-- { result = append(result, matrix[r][left]) }
			left++
		}
	}
	return result
}
```

Complexity Analysis

BRUTE FORCE
Time: O(m*n)
Space: O(m*n)

OPTIMIZED
Time: O(m*n)
Space: O(1) extra

Formula	Memory Tip
4 boundaries: top, bottom, left, right. Each traversal shrinks one boundary. Check mid-traversal for single row/col edge cases.	"Peel the onion: Right, Down, Left, Up. Then shrink. Check boundaries before left/up traversal."

Interview Tips & Edge Cases

The edge case check `if top <= bottom` (before bottom row) and `if left <= right` (before left col) handles single row/column matrices. Don't skip this

---

23

Set Matrix Zeroes

Math & Geometry

Medium

Problem: If an element is 0, set its entire row and column to 0 in-place.

Constraints

m == matrix.length | n == matrix[0].length | 1<=m,n<=200 | In-place

Real-World Use Cases

Spreadsheet propagation, database NULL cascade operations, circuit breaker patterns in distributed systems.

■ What If This Pattern Didn't Exist?

You'd need extra O(m+n) space to mark which rows/cols have zeros. Optimal uses first row/col as markers.

■ Memory Analogy

Mark before acting. Don't zero out rows/cols immediately (cascade problem). First collect all zero coordinates, then propagate. Like the 'mark then

■ Brute Force Approach

Pseudo Code

```
rows = set of rows containing 0
cols = set of cols containing 0
FOR each cell (i,j) with value 0:
   add i to rows, j to cols
FOR each row in rows: zero out row
FOR each col in cols: zero out col
```

Go — Brute Force

```go
func setZeroes(matrix [][]int) {
	m, n := len(matrix), len(matrix[0])
	zeroRows, zeroCols := map[int]bool{}, map[int]bool{}
	for i := 0; i < m; i++ {
		for j := 0; j < n; j++ {
			if matrix[i][j] == 0 { zeroRows[i]=true; zeroCols[j]=true }
		}
	}
	for i := 0; i < m; i++ {
		for j := 0; j < n; j++ {
			if zeroRows[i] || zeroCols[j] { matrix[i][j] = 0 }
		}
	}
}
```

■ Optimized Approach

Pseudo Code

```
// Use first row and first col as markers
firstRowZero = any zero in first row
firstColZero = any zero in first col
FOR i=1 to m-1, j=1 to n-1:
   IF matrix[i][j]==0: matrix[i][0]=0; matrix[0][j]=0
FOR i=1 to m-1, j=1 to n-1:
   IF matrix[i][0]==0 OR matrix[0][j]==0: matrix[i][j]=0
Handle first row and col using saved booleans
```

Go — Optimized

```go
func setZeroes(matrix [][]int) {
	m, n := len(matrix), len(matrix[0])
	firstRow, firstCol := false, false
	for j := 0; j < n; j++ { if matrix[0][j]==0 { firstRow=true } }
	for i := 0; i < m; i++ { if matrix[i][0]==0 { firstCol=true } }
	for i := 1; i < m; i++ {
		for j := 1; j < n; j++ {
			if matrix[i][j] == 0 { matrix[i][0]=0; matrix[0][j]=0 }
		}
	}
	for i := 1; i < m; i++ {
		for j := 1; j < n; j++ {
			if matrix[i][0]==0 || matrix[0][j]==0 { matrix[i][j]=0 }
		}
	}
	if firstRow { for j:=0;j<n;j++ { matrix[0][j]=0 } }
	if firstCol { for i:=0;i<m;i++ { matrix[i][0]=0 } }
}
```

Complexity Analysis

BRUTE FORCE

Time: O(m*n)

Space: O(m+n)

OPTIMIZED

Time: O(m*n)

Space: O(1)

Formula	Memory Tip
First row/col act as marker arrays. Must check if first row/col themselves had zeros BEFORE using them as markers.	"Save first row/col status first. Use them as markers. Apply. Restore first row/col last."

Interview Tips & Edge Cases

Order matters! Save firstRow and firstCol booleans BEFORE using them as markers. Process inner cells (i>=1, j>=1) in the marking pass to avoid

---

24

Happy Number

Math & Geometry

Easy

Problem: A happy number: repeatedly replace n with sum of squares of digits until it reaches 1 (happy) or enters a cycle (not happy).

Constraints

1 <= n <= 2^31 - 1

Real-World Use Cases

Cycle detection pattern used in: Floyd's algorithm for linked list cycles, period detection in simulations, hash collision detection.

■ What If This Pattern Didn't Exist?

Without cycle detection, you'd run forever on unhappy numbers. Hash set detects visited states in O(n) space. Floyd's tortoise-hare: O(1) space.

■ Memory Analogy

Two runners on a circular track. If there's a loop (unhappy), the fast runner eventually laps the slow one — they meet. If there's no loop and the pa

■ Brute Force Approach

Pseudo Code

```
seen = hash set
WHILE n != 1:
   IF n in seen: RETURN false
   seen.add(n)
   n = sumOfSquaredDigits(n)
RETURN true
```

Go — Brute Force

```go
func isHappy(n int) bool {
	sumSq := func(x int) int {
		s := 0
		for x > 0 { d := x % 10; s += d*d; x /= 10 }
		return s
	}
	seen := map[int]bool{}
	for n != 1 {
		if seen[n] { return false }
		seen[n] = true
		n = sumSq(n)
	}
	return true
}
```

■ Optimized Approach

Pseudo Code

```
// Floyd's cycle detection
slow = sumSq(n), fast = sumSq(sumSq(n))
WHILE slow != fast:
   slow = sumSq(slow)
   fast = sumSq(sumSq(fast))
RETURN slow == 1
```

Go — Optimized

```go
func isHappy(n int) bool {
	sumSq := func(x int) int {
		s := 0
		for x > 0 { d := x%10; s+=d*d; x/=10 }
		return s
	}
	slow, fast := n, sumSq(n)
	for slow != fast {
		slow = sumSq(slow)
		fast = sumSq(sumSq(fast))
	}
	return slow == 1
}
```

Complexity Analysis

BRUTE FORCE
Time: O(log n)
Space: O(log n)

OPTIMIZED
Time: O(log n)
Space: O(1)

Formula	Memory Tip
sumOfSquaredDigits(n): d=n%10; sum+=d*d; n/=10. Floyd's: slow moves 1 step, fast moves 2 steps.	"Tortoise and hare on a digit track. If they meet at 1 = happy. If they meet elsewhere = cycle = unhappy."

Interview Tips & Edge Cases

All unhappy numbers eventually cycle through 4. You can hardcode this: if n==4, return false. Floyd's is the elegant O(1) space answer.

25

Plus One

Math & Geometry

Easy

Problem: Given large integer as digit array, increment by one and return the result array.

Constraints

1 <= digits.length <= 100 | No leading zeros | 0 <= digits[i] <= 9

Real-World Use Cases

Big integer arithmetic libraries, counter increment in distributed systems, cryptographic nonce increment.

What If This Pattern Didn't Exist?

Converting to int64 overflows for large numbers. Digit-by-digit handling supports arbitrary precision.

Memory Analogy

Like a car odometer rolling over. Start from the rightmost digit. Add 1. If it becomes 10, set to 0 and carry 1 left. Repeat. If you've carried past all di

Brute Force Approach

Pseudo Code

```
// Convert to number (overflows for large input)
num = int(digits)
RETURN digits of (num + 1)
```

Go — Brute Force

```go
// Overflow risk for large inputs!
func plusOneBrute(digits []int) []int {
	// This FAILS for digits.length > 18
	n := 0
	for _, d := range digits { n = n*10 + d }
	n++
	result := []int{}
	for n > 0 { result = append([]int{n%10}, result...); n /= 10 }
	return result
}
```

Optimized Approach

Pseudo Code

```
FOR i = len-1 down to 0:
   IF digits[i] < 9: digits[i]++; RETURN
   digits[i] = 0  // carry
// All digits were 9: 999 -> 1000
PREPEND 1 to digits
```

Go — Optimized

```go
func plusOne(digits []int) []int {
	for i := len(digits) - 1; i >= 0; i-- {
		if digits[i] < 9 {
			digits[i]++
			return digits
		}
		digits[i] = 0 // carry
	}
	// All 9s: 999... -> 1000...
	return append([]int{1}, digits...)
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n)

Space: O(n)

OPTIMIZED

Time: O(n)

Space: O(1) — usually O(1), O(n) worst case for all 9s

Formula	Memory Tip
Traverse right-to-left. digit < 9: increment and return. digit == 9: set 0 and carry. After loop: prepend 1.	"Odometer rollover. 9 becomes 0 with carry. If all 9s, prepend a 1."

Interview Tips & Edge Cases

Most test cases hit the O(1) early return. Worst case is all 9s (e.g., [9,9,9] -> [1,0,0,0]). Clean solution with early return is preferred in interviews.

---

26 Pow(x, n)

Math & Geometry

Medium

Problem: Implement pow(x, n) — x raised to power n.

Constraints

-100.0 < x < 100.0 | -2^31 <= n <= 2^31-1 | Results within 10^4 range

Real-World Use Cases

RSA encryption (modular exponentiation), compound interest calculations, matrix exponentiation in ML.

What If This Pattern Didn't Exist?

Naive: multiply x by itself n times = O(n). For n=2^31 that's 2 billion multiplications. Fast exponentiation: O(log n).

Memory Analogy

Divide and conquer on exponents. x^8 = (x^4)^2 = ((x^2)^2)^2. You only need 3 multiplications instead of 7. Every time you halve the exponent, yo

Brute Force Approach

Pseudo Code

```
result = 1.0
FOR i = 0 to abs(n)-1:
   result *= x
IF n < 0: result = 1/result
RETURN result
```

Go — Brute Force

```go
func myPowBrute(x float64, n int) float64 {
      if n < 0 { x = 1/x; n = -n }
      result := 1.0
      for i := 0; i < n; i++ { result *= x }
      return result
}
```

Optimized Approach

Pseudo Code

```
IF n < 0: x = 1/x, n = -n
result = 1.0
WHILE n > 0:
   IF n is odd: result *= x
   x = x * x  // square the base
   n = n / 2  // halve the exponent
RETURN result
```

Go — Optimized

```go
func myPow(x float64, n int) float64 {
      if n < 0 { x = 1 / x; n = -n }
      result := 1.0
      for n > 0 {
            if n%2 == 1 { result *= x }
            x *= x
            n /= 2
      }
      return result
}
```

Complexity Analysis

BRUTE FORCE
Time: O(n)
Space: O(1)

OPTIMIZED
Time: O(log n)
Space: O(1)

Formula	Memory Tip
Binary exponentiation: if n odd: result *= x. Always: x=x*x, n=n/2.	"Even power? Just square. Odd power? Grab one x first, then square. Halve n each step."

Interview Tips & Edge Cases

Handle n = INT_MIN carefully: -(-2^31) overflows int32. Use int64 or detect before negating. Recursive version also O(log n) but uses O(log n) sta

---

27

Multiply Strings

Math & Geometry

Medium

Problem: Multiply two non-negative integers represented as strings, return product as string.

Constraints

1 <= num1.length, num2.length <= 200 | Contain digits only | No leading zeros except '0'

Real-World Use Cases

Big integer multiplication (blockchain, cryptography), arbitrary precision arithmetic libraries, scientific computing.

■ What If This Pattern Didn't Exist?

int64 overflows for 200-digit numbers. String multiplication simulates grade-school algorithm on digit arrays.

■ Memory Analogy

Long multiplication from school. Each digit of num2 multiplies each digit of num1. Products overlap at offset positions (i+j and i+j+1). Just like writin

■ Brute Force Approach

Pseudo Code

```
// Convert to Python BigInt (language-dependent)
// Not valid in Go without big.Int
// Simulate with big.Int
```

Go — Brute Force

```go
import (
	"math/big"
)
func multiplyBrute(num1 string, num2 string) string {
	a, _ := new(big.Int).SetString(num1, 10)
	b, _ := new(big.Int).SetString(num2, 10)
	return new(big.Int).Mul(a, b).String()
}
```

■ Optimized Approach

Pseudo Code

```
result = array of (m+n) zeros
FOR i = m-1 down to 0:
   FOR j = n-1 down to 0:
      mul = (num1[i]-'0') * (num2[j]-'0')
      p1 = i+j, p2 = i+j+1
      sum = mul + result[p2]
      result[p2] = sum % 10
      result[p1] += sum / 10
BUILD string from result, skip leading zeros
```

Go — Optimized

```go
func multiply(num1 string, num2 string) string {
	m, n := len(num1), len(num2)
	pos := make([]int, m+n)
	for i := m-1; i >= 0; i-- {
		for j := n-1; j >= 0; j-- {
			mul := int(num1[i]-'0') * int(num2[j]-'0')
			p1, p2 := i+j, i+j+1
			sum := mul + pos[p2]
			pos[p2] = sum % 10
			pos[p1] += sum / 10
		}
	}
	result := []byte{}
	for _, d := range pos {
		if !(len(result)==0 && d==0) { result=append(result, byte('0'+d)) }
	}
	if len(result) == 0 { return "0" }
	return string(result)
}
```

Complexity Analysis

BRUTE FORCE
Time: O(m*n)
Space: O(m+n)

OPTIMIZED
Time: O(m*n)
Space: O(m+n)

Formula	Memory Tip
digit[i] * digit[j] contributes to positions i+j (carry) and i+j+1 (unit) in result array.	"Position formula: digit at i*j lands at pos[i+j+1]. Carry goes to pos[i+j]." Max result length = m+n digits.

Interview Tips & Edge Cases

Edge case: num1='0' or num2='0' → return '0'. Skip leading zeros when building result string. This same pattern is used in polynomial multiplicatio

---

28 Detect Squares

Math & Geometry

Medium

Problem: Design data structure to add points and count axis-aligned squares containing a given query point.

Constraints

Points are non-negative integers. Up to 3000 calls total.

Real-World Use Cases

Computational geometry, collision detection in game engines, spatial indexing, computer vision.

■ What If This Pattern Didn't Exist?

Brute force: O(n^3) for each query. Using point frequency maps: O(n) per query.

■ Memory Analogy

For a query point P, pick any point A on the same column as P. The side length is |P.y - A.y|. Then check if the other two corners (B and C) exist i

■ Brute Force Approach

Pseudo Code

```
// O(n^3) per query
FOR each pair of points A, B:
   IF they form valid square with query point Q:
      count++
```

Go — Brute Force

```go
// Brute force — check all triples
type DetectSquaresBrute struct { points [][2]int }
func (d *DetectSquaresBrute) Add(point []int) {
      d.points = append(d.points, [2]int{point[0], point[1]})
}
// O(n^2) per query using collinear pair approach
```

■ Optimized Approach

Pseudo Code

```
// Data: pointCount map, xPoints map (x -> set of y vals)
ADD(x, y): pointCount[(x,y)]++, xPoints[x].add(y)

COUNT(px, py):
   FOR each y in xPoints[px] (same x as query):
      IF y == py: skip
      side = |y - py|
      // Two possible squares: left and right
      FOR dx in [-side, +side]:
         nx = px + dx
         count += pointCount[(nx,py)] * pointCount[(nx,y)]
```

Go — Optimized

```go
type DetectSquares struct {
	pointCnt map[[2]int]int
	xPoints  map[int]map[int]bool
}
func Constructor() DetectSquares {
	return DetectSquares{
		pointCnt: make(map[[2]int]int),
		xPoints:  make(map[int]map[int]bool),
	}
}
func (d *DetectSquares) Add(point []int) {
	p := [2]int{point[0], point[1]}
	d.pointCnt[p]++
	if d.xPoints[point[0]] == nil { d.xPoints[point[0]] = make(map[int]bool) }
	d.xPoints[point[0]][point[1]] = true
}
func (d *DetectSquares) Count(point []int) int {
	px, py := point[0], point[1]
	count := 0
	for y := range d.xPoints[px] {
		if y == py { continue }
		side := y - py
		for _, dx := range []int{side, -side} {
			nx := px + dx
			count += d.pointCnt[[2]int{nx, py}] * d.pointCnt[[2]int{nx, y}]
		}
	}
	return count
}
```

Complexity Analysis

BRUTE FORCE

Time: O(n^3) per query

Space: O(n)

OPTIMIZED

Time: O(n) per query

Space: O(n)

Formula	Memory Tip
Fix query point P and a same-column point A. Side = |A.y - P.y|. Check corners at (P.x ± side, P.y) and (P.x ± side, A.y). count += freq(corner1) * freq(corner2).	"Fix one side (same x column). The side length determines the square. Check both left and right squares."

Interview Tips & Edge Cases

Multiply point counts (not just bool) because there can be duplicate points. This handles the 'same point added multiple times' case correctly.